

OSSP-PRACTICAL-WEEK11

TASK1: Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }
    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Practical Week 11 - Task 1\n");
    printf("-----\n");

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL,
                      increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n",
          NUM_THREADS * ITERATIONS);

    printf("Actual counter value: %d\n", counter);

    return 0;
}
```

Output:

```
root@LAPTOP-1COJKKG6:~# nano practical_week11_task1.c
root@LAPTOP-1COJKKG6:~# gcc practical_week11_task1.c -o practical_week11_task1 -pthread
root@LAPTOP-1COJKKG6:~# ./practical_week11_task1
Practical Week 11 - Task 1
-----
Expected counter value: 400000
Actual counter value: 281902
```

Observation: I learned how to create and manage POSIX threads using pthread_create() and pthread_join(). I understood how multiple threads access a shared variable concurrently and how race conditions can occur when threads update the same variable without synchronization.

Task2: Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

Source code:

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;
pthread_mutex_t lock;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    pthread_mutex_init(&lock, NULL);

    printf("Practical Week 11 - Task 2\n");
    printf("-----\n");

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL,
                      increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n",
          NUM_THREADS * ITERATIONS);

    printf("Actual counter value: %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

Output:

```
root@LAPTOP-1COJKKG6:~# nano practical_week11_task2.c
root@LAPTOP-1COJKKG6:~# gcc practical_week11_task2.c -o practical_week11_task2 -pthread
root@LAPTOP-1COJKKG6:~# ./practical_week11_task2
Practical Week 11 - Task 2
-----
Expected counter value: 400000
Actual counter value: 400000
root@LAPTOP-1COJKKG6:~# |
```

Observation: I learned how to use mutex locks with `pthread_mutex_lock()` and `pthread_mutex_unlock()` to protect shared data. I understood how synchronization prevents race conditions and ensures that the counter produces the expected result. I also learned how mutexes improve data consistency in multithreaded applications.